

Function, Scope and Hoisting

Function

→ A function is a reusable block of code that perform a specific task.

Syntax - $\left[\begin{array}{l} \text{function } \underline{\text{name}}() \{ \\ \quad \text{// code} \\ \} \end{array} \right] \underline{\text{Body}}$

$\underbrace{\text{name}()}_{\text{Function call.}}$

Function with parameter

→ A function can accept value → parameter

$\left[\begin{array}{l} \text{function } \underline{\text{greet}}(\underline{\text{name}}) \{ \\ \quad \text{Console.log("Hello", name)} \\ \} \end{array} \right]$

$\downarrow$ $\text{greet}(\underline{\text{"Satyam"}})$

Parameter

4,
Hello Satyam

→ A function can return a value using the return keyword.

let result = add(10, 20)

Types of function -

- New Section 1 Page 2

- ⑤ JSFE
- ⑥ callback function
- ⑦ HOF
- ⑧ Recursive f

① Function Declaration

→ This is the traditional way of creating a function

```

function add(a, b) {
    return a + b
}
  
```

hoisted

② Function Expression ✓

→ A function can be assigned to a variable

```

const greet = function () {
    console.log("Hello")
}
  
```

greet() ✓

1. ⑨ Anonymous function -

→ A function without a name is called an anonymous function

```
const greet = function () {  
    console.log("Hello")  
}
```

⑩ Arrow function -

→ Arrow functions were introduced in modern JS as a shorter way to write functions.

Normal function

```
function add(a, b) {  
    return a + b  
}
```

⇒ Arrow function

```
const add = (a, b) => {  
    return a + b  
}
```

↓
console.log(add(10, 30))

→ if the function contains only one expression, we can make it shorter

$$[\text{const add} = (a, b) \Rightarrow a + b]$$

⑤ IIFE → Immediately invoked function expression -

→ It is a function that executes immediately after it is created

Syntax
$$\left[\begin{array}{l} (\text{function } () \{ \\ \quad \text{console.log("Hello Duniya")} \\ \})() ; \end{array} \right]$$

⑥ Callback function -

→ A function passed as an argument to another function is called a callback function.

```
function greet (name) {
  console.log("Hello", name)
}
```

```
function processUser (callback) {
  callback ("Satyam")
}
```

→ processUser (greet) → greet is passed to processUser

↑ callback

⑦ HOF → higher order function

→ A function that accepts another function as an argument or, returns another function.

```
function calculate (a, b, operation) {
  return operation (a, b)
}
```

calling -

```
function add (x, y) {
  return x + y
}
```

}

→ hoisting is JS behaviour of processing declaration before executing the code.

var let a = 10 } → a → undefined

console.log(a)

Function declaration hoisting

→ fully hoisted -

→ The function can be called before its declaration.

greet()

function greet() {
 console.log("Satyam")
}

TOP

Function Expression hoisting

greet()

const greet = function () {
 console.log("Hello Duniya")
}

Error
greet is
not defined

→ The result is an error because
greet can not be accessed before
the const variable is initialized



Same for Arrow function